Assignment 5

```
Name : Ravish Ranjan
Roll no. : 45
Course : MCA
Subject : Information Security
```

Implement these 2 practicals in Python (or any suitable programming language) in a single program, and attach screenshots showing the source code and the corresponding output for each practical:

1. Cryptanalysis of the Additive Cipher
2. Cryptanalysis of the Multiplicative Cipher

a4.h

```h
#pragma once

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char* additive_cipher(const char* data, int key);
char* multiplicative_cipher(const char* data, int key);
char* additive_decipher(const char* data, int key);
char* multiplicative_decipher(const char* data, int key);
```

a4.c

```c
#include "a4.h"

int extended_euclidean(int a, int b, int* x, int* y) {
    if (b == 0) {
        *x = 1;
        *y = 0;
        return a;
    }

    int x1,y1;

    int gcd = extended_euclidean(b,a%b,&x1,&y1);

    *x = y1;
    *y = x1 - (a/b)*y1;
```

```
    return gcd;
}

char* additive_cipher(const char* data, int key) {
    char* output = (char*)calloc(strlen(data)+1,sizeof(char));

    int at = 0;
    while (data[at] != '\0') {
        int c = data[at];

        if (
            ((c >= 'a') && (c <= 'z')) ||
            ((c >= 'A') && (c <= 'Z'))
        ) {
            c += (key%26);

            if (
                ((data[at] >= 'a') && (data[at] <= 'z') && (c > 'z')) ||
                ((data[at] >= 'A') && (data[at] <= 'Z') && (c > 'Z'))
            ) {
                c -= 26;
            } else if (
                ((data[at] >= 'a') && (data[at] <= 'z') && (c < 'a')) ||
                ((data[at] >= 'A') && (data[at] <= 'Z') && (c < 'A'))
            ) {
                c += 26;
            }
        } else if ((c >= '0') && (c <= '9')) {
            c += (key%10);
            if (c > '9') c -= 10;
            else if (c < '0') c += 10;
        }
        output[at++] = (char)c;
    }
    output[at] = '\0';

    return output;
}

char* multiplicative_cipher(const char* data, int key) {
    char* output = (char*)calloc(strlen(data)+1,sizeof(char));
    output[strlen(data)] = '\0';

    key %= 26;
    if (key < 0) key += 26;

    int x,y;

    if (extended_euclidean(key,26,&x,&y) != 1) {
        printf("Error : Not a valid key\n");
        return output;
    }

    int at = 0;
```

```
    while (data[at] != '\0') {
        int c = data[at];

        if ((c >= 'a') && (c <= 'z')) {
            c = 'a' + ((c - 'a')*key)%26;
        } else if ((c >= 'A') && (c <= 'Z')) {
            c = 'A' + ((c - 'A')*key)%26;
        }

        output[at++] = (char)c;
    }
    output[at] = '\0';

    return output;
}

char* additive_decipher(const char* data, int key) {
    return additive_cipher(data,-key);
}

char* multiplicative_decipher(const char* data, int key) {
    int x,y;
    key %= 26;
    int gcd = extended_euclidean(key,26,&x,&y);

    char* output = (char*)calloc(strlen(data)+1,sizeof(char));
    output[strlen(data)] = '\0';

    if (gcd != 1) {
        printf("Multiplicative cipher Error : Not a valid key\n");
        return output;
    }

    return multiplicative_cipher(data,(x%26 + 26)%26);
}

// int main() {
//     char* data = "Hello, World! This is Ravish Ranjan in 2026: Cipher
Test #1. Keep it Safe & Simple!";
//     int key = 75;

//     char* add_cipher = additive_cipher(data,key);
//     char* add_decipher = additive_decipher(add_cipher,key);
//     char* mul_cipher = multiplicative_cipher(data,key);
//     char* mul_decipher = multiplicative_decipher(mul_cipher,key);

//     printf("\nData     : %s\n\n",data);
//     printf("Cipher   : %s\n",add_cipher);
//     printf("Decipher : %s\n",add_decipher);
//     printf("Is Same  : %s\n\n",strcmp(add_decipher,data) == 0?
"Yes":"No");

//     printf("Cipher   : %s\n",mul_cipher);
//     printf("Decipher : %s\n",mul_decipher);
```

```
//     printf("Is Same  : %s\n\n",strcmp(mul_decipher,data) ==
0?"Yes":"No");
//     return 0;
// }
```

a5.c

```
#include "a4.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void cryptanalysis_additive(const char* ciphertext) {
    printf("1. Cryptanalysis of Additive Cipher\n");
    printf("Ciphertext: %s\n\n", ciphertext);
    printf("Trying all possible keys (0 to 25):\n\n");

    char* plaintext = (char*)calloc(500,sizeof(char));
    for (int k = 0; k < 26; k++) {
        plaintext = additive_decipher(ciphertext, k);
        printf("Key %2d : %s\n", k, plaintext);
    }
    printf("\n");
}

void cryptanalysis_multiplicative(const char* ciphertext) {
    printf("2. Cryptanalysis of Multiplicative Cipher\n");
    printf("Ciphertext: %s\n\n", ciphertext);
    printf("Trying all valid multiplicative keys (coprime to 26):\n");

    char* plaintext = (char*)calloc(500,sizeof(char));
    int valid_keys[] = {1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25};
    int num_keys = sizeof(valid_keys) / sizeof(valid_keys[0]);

    for (int i = 0; i < num_keys; i++) {
        int k = valid_keys[i];
        plaintext = multiplicative_decipher(ciphertext, k);
        printf("Key %2d : %s\n", k, plaintext);
    }
    printf("\n");
}

int main() {
    const char* add_ciphertext = "Ebiil, Tloia! Qefp fp Oxsfpe Oxkgxk fk
7571: Zfmebo Qbpq #6. Hbbm fq Pxcb & Pfjmib!";
    const char* mul_ciphertext = "Fottk, Mkbtr! Vfcy cy Bapcyf Banzan cn
2026: Uchfob Voyv #1. Wooh cv Yalo & Ycqhto!";

    cryptanalysis_additive(add_ciphertext);
    cryptanalysis_multiplicative(mul_ciphertext);
```

```
    return 0;
}
```

```
ravish@ravishPC:../is/prog ⎇ (main)> gcc -c ./a4.c -o a4.o
ravish@ravishPC:../is/prog ⎇ (main)> gcc -c ./a5.c -o a5.o
ravish@ravishPC:../is/prog ⎇ (main)> gcc ./a4.o ./a5.o -o ./a5.out
ravish@ravishPC:../is/prog ⎇ (main)> ./a5.out
1. Cryptanalysis of Additive Cipher
Ciphertext: Ebiil, Tloia! Qefp fp Oxsfpe Oxkgxk fk 7571: Zfmebo Qbpq #6.
Hbbm fq Pxcb & Pfjmib!

Trying all possible keys (0 to 25):

Key  0 : Ebiil, Tloia! Qefp fp Oxsfpe Oxkgxk fk 7571: Zfmebo Qbpq #6. Hbbm
fq Pxcb & Pfjmib!
Key  1 : Dahhk, Sknhz! Pdeo eo Nwreod Nwjfwj ej 6460: Yeldan Paop #5. Gaal
ep Owba & Oeilha!
Key  2 : Czggj, Rjmgy! Ocdn dn Mvqdnc Mvievi di 5359: Xdkczm Ozno #4. Fzzk
do Nvaz & Ndhkgz!
Key  3 : Byffi, Qilfx! Nbcm cm Lupcmb Luhduh ch 4248: Wcjbyl Nymn #3. Eyyj
cn Muzy & Mcgjfy!
Key  4 : Axeeh, Phkew! Mabl bl Ktobla Ktgctg bg 3137: Vbiaxk Mxlm #2. Dxxi
bm Ltyx & Lbfiex!
Key  5 : Zwddg, Ogjdv! Lzak ak Jsnakz Jsfbsf af 2026: Uahzwj Lwkl #1. Cwwh
al Ksxw & Kaehdw!
Key  6 : Yvccf, Nficu! Kyzj zj Irmzjy Ireare ze 1915: Tzgyvi Kvjk #0. Bvvg
zk Jrwv & Jzdgcv!
Key  7 : Xubbe, Mehbt! Jxyi yi Hqlyix Hqdzqd yd 0804: Syfxuh Juij #9. Auuf
yj Iqvu & Iycfbu!
Key  8 : Wtaad, Ldgas! Iwxh xh Gpkxhw Gpcypc xc 9793: Rxewtg Ithi #8. Ztte
xi Hput & Hxbeat!
Key  9 : Vszzc, Kcfzr! Hvwg wg Fojwgv Fobxob wb 8682: Qwdvsf Hsgh #7. Yssd
wh Gots & Gwadzs!
Key 10 : Uryyb, Jbeyq! Guvf vf Enivfu Enawna va 7571: Pvcure Grfg #6. Xrrc
vg Fnsr & Fvzcyr!
Key 11 : Tqxxa, Iadxp! Ftue ue Dmhuet Dmzvmz uz 6460: Oubtqd Fqef #5. Wqqb
uf Emrq & Euybxq!
Key 12 : Spwwz, Hzcwo! Estd td Clgtds Clyuly ty 5359: Ntaspc Epde #4. Vppa
te Dlqp & Dtxawp!
Key 13 : Rovvy, Gybvn! Drsc sc Bkfscr Bkxtkx sx 4248: Mszrob Docd #3. Uooz
sd Ckpo & Cswzvo!
Key 14 : Qnuux, Fxaum! Cqrb rb Ajerbq Ajwsjw rw 3137: Lryqna Cnbc #2. Tnny
rc Bjon & Brvyun!
Key 15 : Pmttw, Ewztl! Bpqa qa Zidqap Zivriv qv 2026: Kqxpmz Bmab #1. Smmx
qb Ainm & Aquxtm!
Key 16 : Olssv, Dvysk! Aopz pz Yhcpzo Yhuqhu pu 1915: Jpwoly Alza #0. Rllw
pa Zhml & Zptwsl!
Key 17 : Nkrru, Cuxrj! Znoy oy Xgboyn Xgtpgt ot 0804: Iovnkx Zkyz #9. Qkkv
oz Yglk & Yosvrk!
Key 18 : Mjqqt, Btwqi! Ymnx nx Wfanxm Wfsofs ns 9793: Hnumjw Yjxy #8. Pjju
```

```
ny Xfkj & Xnruqj!
Key 19 : Lipps, Asvph! Xlmw mw Vezmwl Verner mr 8682: Gmtliv Xiwx #7. Oiit
mx Weji & Wmqtpi!
Key 20 : Khoor, Zruog! Wklv lv Udylvk Udqmdq lq 7571: Flskhu Whvw #6. Nhhs
lw Vdih & Vlpsoh!
Key 21 : Jgnnq, Yqtnf! Vjku ku Tcxkuj Tcplcp kp 6460: Ekrjgt Vguv #5. Mggr
kv Uchg & Ukorng!
Key 22 : Ifmmp, Xpsme! Uijt jt Sbwjti Sbokbo jo 5359: Djqifs Uftu #4. Lffq
ju Tbgf & Tjnqmf!
Key 23 : Hello, World! This is Ravish Ranjan in 4248: Cipher Test #3. Keep
it Safe & Simple!
Key 24 : Gdkkn, Vnqkc! Sghr hr Qzuhrg Qzmizm hm 3137: Bhogdq Sdrs #2. Jddo
hs Rzed & Rhlokd!
Key 25 : Fcjjm, Umpjb! Rfgq gq Pytgqf Pylhyl gl 2026: Agnfcp Rcqr #1. Iccn
gr Qydc & Qgknjc!

2. Cryptanalysis of Multiplicative Cipher
Ciphertext: Fottk, Mkbtr! Vfcy cy Bapcyf Banzan cn 2026: Uchfob Voyv #1.
Wooh cv Yalo & Ycqhto!

Trying all valid multiplicative keys (coprime to 26):
Key  1 : Fottk, Mkbtr! Vfcy cy Bapcyf Banzan cn 2026: Uchfob Voyv #1. Wooh
cv Yalo & Ycqhto!
Key  3 : Twppm, Emjpx! Htsi si Jafsit Janran sn 2026: Ysltwj Hwih #1. Qwwl
sh Iavw & Isolpw!
Key  5 : Bijjc, Scvjt! Zbqk qk Vadqkb Vanfan qn 2026: Eqrbiv Zikz #1. Uiir
qz Kaxi & Kqyrji!
Key  7 : Xczzu, Yupzv! Dxew ew Parewx Panlan en 2026: Oebxcp Dcwd #1. Sccb
ed Wajc & Wegbzc!
Key  9 : Pqffe, Kedfz! Lpgu gu Datgup Danxan gn 2026: Igvpqd Lqul #1. Oqqv
gl Uahq & Ugwvfq!
Key 11 : Rgxxi, Uitxl! Jrmo mo Tazmor Tanhan mn 2026: Qmdrgt Jgoj #1. Cggd
mj Oabg & Omsdxg!
Key 15 : Judds, Gshdp! Rjom om Habomj Hantan on 2026: Koxjuh Rumr #1. Yuux
or Mazu & Moixdu!
Key 17 : Lkvvw, Qwxvb! Plug ug Xahugl Xandan un 2026: Suflkx Pkgp #1. Mkkf
up Gatk & Guefvk!
Key 19 : Dybbg, Cglbf! Xdwe we Lajwed Lanpan wn 2026: Mwzdyl Xyex #1. Iyyz
wx Eary & Ewuzby!
Key 21 : Zsrry, Iyfrh! Bzkq kq Faxkqz Fanvan kn 2026: Wkjzsf Bsqb #1. Gssj
kb Qads & Qkcjrs!
Key 23 : Hello, World! This is Ravish Ranjan in 2026: Cipher Test #1. Keep
it Safe & Simple!
Key 25 : Vmhhq, Oqzhj! Fvyc yc Zalycv Zanban yn 2026: Gytvmz Fmcf #1. Emmt
yf Capm & Cykthm!

ravish@ravishPC:../is/prog ⎇ (main)>
```

Conclusion

1. in additive decipher with key = 23 a meaningful message is retrieved (not completly , number is not correctly deciphered cause the key has to be multiple of 23 and 10 both because of my implementaion)

```
Key 23 : Hello, World! This is Ravish Ranjan in 4248: Cipher Test #3. Keep
it Safe & Simple!
```

2. in multiplicative decipher too with key = 23 a meaningful message is retrieved

```
Key 23 : Hello, World! This is Ravish Ranjan in 2026: Cipher Test #1. Keep
it Safe & Simple!
```